

EXAMEN • PRIMER PARCIAL

Universidad Autónoma Tomás Frías
Facultad de Ciencias Puras · Carrera de Ingeniería Informática

Programación Orientada a Objetos

INF 220 · Unidad 1 — Clases, objetos, constructores y colecciones

gestión 01/2026 · modalidad entrega + defensa · puntaje total 100 pts · lenguaje Java (JDK 25)

INSTRUCCIONES

Resuelva en Java. Se evalúa la correcta declaración de clases, el encapsulamiento y el uso de constructores, métodos, manejo de cadenas y colecciones. Escriba código legible e indentado; incluya el método main solicitado en cada ejercicio y los resultados de su ejecución. El trabajo es individual; la autoría se verificará en la defensa.

ENTREGA Y DEFENSA

La resolución se entrega en un único documento PDF con las soluciones (código Java y resultados de ejecución) a través de la **plataforma virtual de la universidad**, dentro del plazo establecido por el docente. Además, cada estudiante realizará una **defensa del trabajo en horario de clases**, requisito indispensable para la validez de la evaluación.

[01] Sensor ambiental**10 pts**

Diseñe la clase SensorAmbiental con los atributos ubicacion (String), temperatura (double) y humedad (double). Agregue un método mostrarLectura() que imprima los tres datos con formato legible y un método estadoAlerta() que devuelva true si la temperatura supera los 35 °C. En main, cree tres sensores de distintas zonas de Potosí y muestre sus lecturas indicando cuáles están en alerta.

[02] Cabina de teleférico**10 pts**

Implemente la clase Cabina con los atributos codigo (String), capacidad (int) y pasajerosActuales (int). Defina tres constructores: uno sin parámetros (valores por defecto: capacidad 8, 0 pasajeros), uno que reciba solo codigo y capacidad, y uno que reciba los tres atributos. Incluya un método disponibilidad() que retorne cuántos lugares quedan libres. Instancie una cabina con cada constructor y muestre su disponibilidad.

[03] Reserva de biblioteca**10 pts**

Cree la clase Prestamo con atributos privados titulo (String), diasPrestamo (int) y devuelto (boolean). Provea getters y setters, pero el setDiasPrestamo debe rechazar valores menores a 1 o mayores a 15 (dejando el valor anterior y avisando por consola). Agregue un método registrarDevolucion() que ponga devuelto en true. Demuestre en main un intento válido y uno inválido de asignar los días.

[04] Registro de vehículos**10 pts**

Diseñe la clase Vehiculo con atributos placa (String) y marca (String), y un atributo estático totalRegistrados que se incremente automáticamente cada vez que se crea un vehículo. Agregue un método estático cantidadRegistrada() que devuelva ese total y un método de instancia descripcion(). Cree cinco vehículos y, sin recorrerlos manualmente, muestre cuántos se registraron.

[05] Calculadora de tarifas**10 pts**

Implemente la clase CalculadoraTarifa con un método calcular sobrecargado: calcular(int minutos) para llamadas telefónicas (0.50 Bs por minuto), calcular(double megas) para datos móviles (2 Bs por mega) y calcular(int minutos, double megas) para un paquete combinado con 10 % de descuento sobre la suma. En main, invoque las tres versiones y muestre cada monto.

[06] Analizador de placas**10 pts**

Escriba la clase AnalizadorPlaca con un método esValida(String placa) que verifique que la placa tenga exactamente 7 caracteres, empezando con 4 dígitos seguidos de 3 letras mayúsculas (ej. 1234ABC). Agregue extraerLetras(String placa) que devuelva solo la porción de letras y normalizar(String placa) que elimine espacios y convierta a mayúsculas. Pruebe con al menos tres placas, una de ellas inválida.

[07] Cola de atención**10 pts**

Usando ArrayList<String>, cree la clase ColaAtencion con métodos agregar(String nombre), atender() (retira y devuelve el primero de la lista), cantidadEnEspera() y mostrarCola(). En main, agregue cinco personas, atienda a dos y muestre quiénes siguen en espera junto con el total restante.

[08] Inventario de minerales**10 pts**

Con HashMap<String, Integer>, implemente la clase Inventario que permita registrar(String mineral, int kilos) (si el mineral ya existe, suma los kilos), consultar(String mineral) y mostrarTodo(). Registre entradas repetidas de al menos dos minerales para comprobar que las cantidades se acumulan correctamente.

[09] Depuración de correos**10 pts**

Reciba un arreglo de correos electrónicos con duplicados y, usando HashSet<String>, escriba un método depurar(String[] correos) que devuelva la cantidad de correos únicos y otro existe(String correo). Muestre la lista final sin repetidos y confirme la búsqueda de un correo presente y uno ausente.

[10] Catálogo de canciones**10 pts**

Defina la clase Cancion con atributos titulo, artista (String) y duracion (int, en segundos), con su constructor completo y un toString() apropiado. Luego cree la clase Catalogo que mantenga un ArrayList<Cancion> con los métodos agregar(Cancion c), buscarPorArtista(String artista) (devuelve todas las canciones de ese artista) y duracionTotal() (suma los segundos de todas). Cargue al menos cuatro canciones y demuestre la búsqueda y la duración total.

// fin del examen – revise su código antes de entregar